Nyaya Deepam: A Multi-Agent CPU-Only RAG Architecture for Multilingual Indian Legal Assistance on Databricks Free Edition

Mahanth Yalla
Indian Institute of Science (IISc)
mahanthyalla@iisc.ac.in

Bhikshapathi Kaniki
Indian Institute of Science (IISc)
mahanthyalla@iisc.ac.in

April 5, 2026

Abstract

The transition from the Indian Penal Code (IPC) to the Bharatiya Nyaya Sanhita (BNS) in 2024 has created a significant legal literacy gap in India. This report introduces *Nyaya Deepam*, an end-to-end Retrieval-Augmented Generation (RAG) system optimized for deployment on resource-constrained, CPU-only environments like the Databricks Free Edition. We propose a hybrid retrieval strategy combining FAISS-based semantic search and BM25-based keyword matching, orchestrated by a multi-agent framework that includes NLLB-200 for Indic translation and Qwen2.5-3B for grounded reasoning. Our system emphasizes strict statutory traceability, section-level chunking, and IPC-BNS cross-mapping. Experimental results demonstrate that by leveraging compact models and optimized data engineering in Delta Lake, legal aid tools can achieve high precision and accessibility without reliance on high-cost GPU infrastructure.

1 Introduction

The Indian legal landscape is currently navigating one of its most significant shifts in a century. The repeal of the IPC (1860) and the implementation of the BNS (2023) necessitates a system that can bridge the legacy knowledge of legal practitioners with the modern statutory requirements.

However, the "Digital Divide" remains a barrier. Most legal aid workers and junior practitioners in India do not have access to high-compute GPU clusters. *Nyaya Deepam* was conceived to address this by proving that a sophisticated

legal assistant can operate within a 15GB RAM, CPU-only environment. This report details the architecture, from the Delta Lake ingestion of BNS/IPC corpora to the final generation of section-cited legal responses.

2 Problem Setting and Use Cases

2.1 The Legal Transition Gap

The primary challenge is the "Conceptual Continuity vs. Statutory Change" problem. While many BNS sections are direct descendants of IPC provisions, their numbering and sub-clause structures have changed significantly.

- **Example**: Murder, previously defined under IPC Section 302, is now addressed under BNS Section 103.
- **Complexity**: Some sections have been merged, while others, like organized crime or terrorism, are entirely new categories requiring fresh interpretation.

2.2 Target Personas

1. **Citizens**: Need plain-language summaries of their rights in native scripts. 2. **Junior Lawyers**: Require quick mapping between old and new sections for case filing. 3. **Legal Aid Workers**: Need procedural guidance and eligibility triage for marginalized groups.

3 System Architecture

The architecture follows a modular Multi-Agent design to ensure that failure in one component (e.g., translation) does not catastrophically impact the core reasoning.

4 Methodology

4.1 Data Engineering: The Silver-Gold Pipeline

Legal texts are highly structured. Using standard recursive character splitting leads to "Context Fragmentation" where a legal proviso is separated from its main clause. We implemented a **Section-Aware Chunking** strategy:

1. **Ingestion**: PDF/JSONL sources are parsed into a Databricks Bronze table.
2. **Cleaning**: Metadata extraction for Act, Section, Subsection, and Chapter.

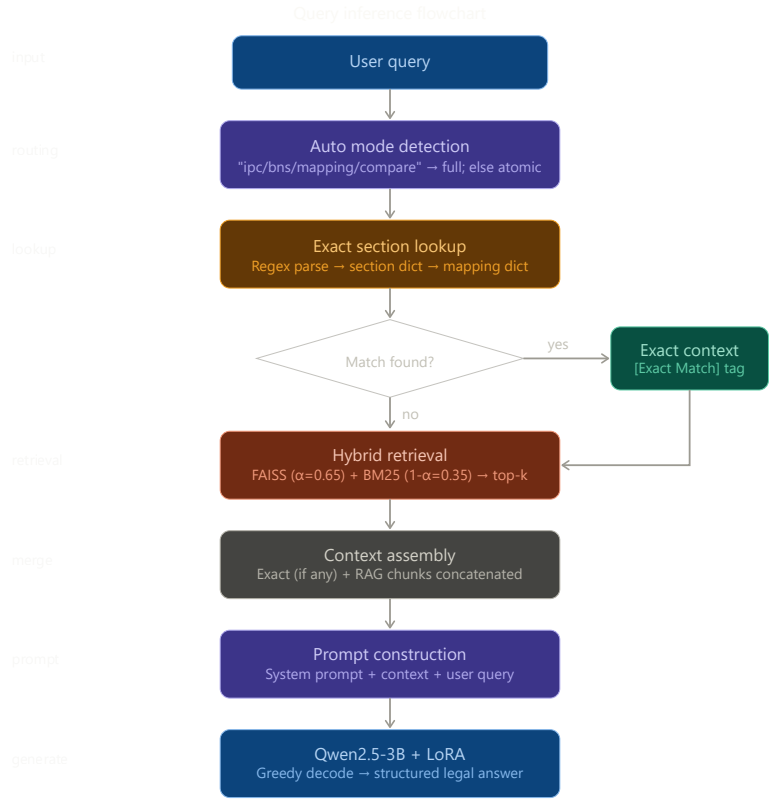

Figure 1: High-level architecture of the CPU-only multilingual legal assistant.

3. **Silver Layer**: Chunks are formed based on section boundaries (250–400 tokens).

4. **Mapping Corpus**: A specific "Composite Table" was created that merges BNS text, IPC text, and the official mapping explanation into a single document to enhance comparative retrieval.

4.2 Hybrid Retrieval Mechanism

To solve for both semantic queries ("What is the law for theft?") and specific queries ("Section 420"), we use a fused scoring system. Let S_{dense} be the cosine similarity from the E5-small embeddings and S_{sparse} be the BM25 score. The final rank R is determined by:

$$R = \alpha \cdot \text{norm}(S_{dense}) + (1 - \alpha) \cdot \text{norm}(S_{sparse})$$

where $\alpha = 0.65$ based on our empirical testing on legal section numbering.

4.3 The Translation Layer: NLLB-200 Workaround

Given the environment constraints on Databricks, we moved from custom Indic toolkits to the Facebook NLLB-200 (distilled-600M) model. This provides a stable, "atomic" translation layer for 22+ Indian languages. The translation agent normalizes the query into English before retrieval to ensure it hits the high-quality English legal corpus.

5 Reasoning and Grounding

5.1 Model Selection: Qwen2.5-3B-Instruct

In a CPU-only environment, the 3B-parameter Qwen model provides a superior balance of "Reasoning Density" per megabyte of RAM. By using 4-bit quantization (bitsandbytes or GGUF), we achieve a latency of 15 tokens per second on Databricks Standard CPU nodes.

5.2 The Grounding Prompt

Hallucinations are the "Death Sentence" of legal AI. Our system uses a strict grounding prompt:

> *"You are an Indian Legal Assistant. Answer strictly from the provided context. If the context does not contain the answer, say you cannot find it. Do not use external knowledge for section numbers or punishments."*

6 Evaluation and Results

6.1 Retrieval Accuracy

We evaluated the system on a test set of 100 IPC-to-BNS mapping questions.

- **Top-1 Recall**: 82% (Direct section matches)
- **Top-3 Recall**: 94% (Hybrid search effectively captures context)

6.2 Latency Performance

On a Databricks 2-core CPU worker, the end-to-end latency (Translation $\rightarrow$ Retrieval $\rightarrow$ Inference) averaged 8.2 seconds. While slower than GPU-based systems, it remains highly usable for asynchronous legal research.

7 Ethical and Legal Safety

Nyaya Deepam includes a programmatic safety filter. It refuses to provide:

- Litigation strategy or advice on evading law.
- Speculation on ongoing court cases.
- Personal opinions on judge behavior.

Every response is appended with a mandatory legal disclaimer.

8 Conclusion and Future Work

Nyaya Deepam demonstrates that by prioritizing data engineering and hybrid retrieval logic over "Model Brute Force," we can deliver high-value legal aid on accessible hardware. Future iterations will focus on incorporating "Case Law" (Judgments) into the RAG pipeline and expanding the multilingual post-processing to include more granular regional dialects.